THE ARCHITECTURE

# From MARTA's feeds to your phone, in five steps.

1 · DATA

## MARTA real-time APIs

Every bus position and every train's arrival predictions, plus the timetable.

→

2 · VULTR

## Recorders, nonstop

A Python script asks MARTA every 15 to 20 s and saves every vehicle.

→

3 · TIGER DATA

## The history

TimescaleDB (when) + PostGIS (where). 580,000+ positions.

→

4 · API

## Match + plan

FastAPI on Vultr finds your exact vehicle and the fastest way back.

→

5 · APP

## bringit.casa

Next.js on Vercel, live MapLibre map, no login.

**Gemini, two jobs only** A spoken sentence into a stop, line and time · rewording the report email. It never decides anything.

VULTR

# A cloud computer that records MARTA around the clock.

MARTA only shows where vehicles are right now. To know which bus passed your stop 10 minutes ago, someone has to record it. That's our Vultr server.




### 2 Python recorders

One for buses (every 15 s), one for trains (every 20 s). Same loop forever.

### Restart themselves

The server's service manager (systemd) starts them on boot and restarts them if they crash.

### Also runs our API

FastAPI behind Caddy (HTTPS): matching, live tracking, the plan.

TIGER DATA · TIMESCALEDB

# Our history is split into 6-hour chunks by time.

A question about 2:40 to 2:50 PM opens one chunk and skips the rest.
Searches stay fast, no matter how big the history gets.

`vehicle_positions (looks like one table: 580,000+ rows)`

CHUNK
**2 AM to 8 AM**
skipped

CHUNK
**8 AM to 2 PM**
skipped

CHUNK
**2 PM to 8 PM**
searched: 2:40 to 2:50

CHUNK
**8 PM to 2 AM**
skipped

**1 chunk**
searched instead of the whole history

Like a library sorted by date: go straight to Friday afternoon's shelf instead of reading every book.

TIGER DATA · COMPRESSION

# Each column is stored shorter. Nothing is lost.

Chunks older than a day compress automatically. Repeats become one value and a count; changing values become small differences.

BEFORE: ROW BY ROW

| TIME | BUS | ROUTE | LATITUDE |
|---|---|---|---|
| 2:43:10 | 1631 | 49 | 33.7712 |
| 2:43:40 | 1631 | 49 | 33.7718 |
| 2:44:10 | 1631 | 49 | 33.7725 |
| 2:44:40 | 1631 | 49 | 33.7731 |

→

AFTER: COLUMN BY COLUMN, PER BUS

**route: 49 × 4**
same value repeated → stored once with a count

**time: 2:43:10, +30 s, +30 s, +30 s**
changing values → stored as small differences

**bus 1631: stored once for the group**
data is grouped per vehicle

**9.6×**
smaller (30 MB → 3.3 MB of older data)

**Lossless**
verified: 57,544 real rows came back identical

**On demand**
rebuilds only the vehicles, minutes and columns a search needs

TIGER DATA · POSTGIS

# When plus where, in one search.

TimescaleDB narrows to the time. PostGIS keeps only vehicles within a quarter mile of your stop.




STEP 1 · TIMESCALEDB

**only 2:35 to 2:55 PM**

STEP 2 · POSTGIS

**only within a quarter mile of North Ave**

STEP 3 · EACH VEHICLE'S TRACK

**when it came closest, and how close**

RESULT

**"Red 406 passed North Ave at 2:46 PM, 40 ft away"**

ACCURATE PREDICTIONS

# MARTA gives one-way arrival times. We measured the rest.

MARTA's API says when a train reaches each station on its current trip, not how long it waits at the end of the line or when it comes back.

1 · RECORD

### Vultr → Tiger Data

Every bus and train, every 15 to 20 s.

→

2 · MEASURE (PYTHON)

### How MARTA really behaves

Trains turn in ~34 s and wait ~7 min at the end. Buses wait ~12 min and come back ~9 in 10. Each end of the line, by line and time of day.

→

3 · BUILD

### The planner's rules

Measured numbers, not the printed schedule.

→

4 · COMPARE

### Rewind and check

Ask the planner at thousands of past moments, then check what the train really did.

**~96%**
trains: first pick was truly the fastest way back

**8.2% → 1.4%**
missed trains after the replay caught our bugs

**8:57 → 8:57:30**
live test: predicted vs actual, 22 min ahead

WHERE THE AI IS

# Gemini translates. Real transit data decides.

JOB 1

## A sentence into answers

"got off at five points from the blue line just now"

→ train · Blue Line · Five Points · about 5 min ago. Our code checks the place against real stops and reads the time itself.

JOB 2

## Wording the report email

Every fact (train, time, place) is checked against our data. If anything changed, the plain template is used instead.

NEVER

## Deciding anything

Matching, tracking and the plan are deterministic code. A model could invent a train; ours can't.

4 s deadline · circuit breaker · daily cap · tap questions always work without it.